

LinkedIn Implementation

23PT01 - Aakash Velusamy

23PT02 - Akash K

23PT14 - Kabilan S

OVERVIEW:

This project examines the role of fundamental data structures—linked lists, stacks, queues, and binary trees—in implementing a social media network system using C++. Each data structure serves a distinct purpose in managing user interactions, connections, posts, and notifications within the network.

DATA STRUCTURES USED:

- Arrays
- Stacks
- Queues
- Linked Lists
- Trees

APPLICATIONS:

- **Linked Lists: Managing User Connections**
 - Linked lists establish and manage user connections, such as followers and those being followed.
 - Each node represents a user, containing user information and a pointer to the next user.
 - Operations like adding and removing followers, as well as traversing the list to display connections, are efficiently performed.
- **Stacks and Queues: Managing Notifications**
 - Stacks manage notifications within the social media network.
 - Notifications are stored in a stack, allowing users to view them in a last-in-first-out (LIFO) order.

-
- Stack operations enable efficient notification storage and retrieval, ensuring chronological access for users.
 - **Binary Trees for Post Management**
 - Binary trees manage user posts within the network.
 - Each user's posts are stored in a binary tree structure, with nodes representing posts arranged in a defined order (e.g., chronological).
 - Binary tree operations such as insertion and traversal enable efficient management and retrieval of user posts, facilitating seamless interaction and content consumption.

Through the effective utilization of these data structures, the social media network system efficiently manages user interactions, connections, posts, and notifications, enhancing user experience and system performance.

REFERENCES:

- [Queue Linked List Implementation](#)
- [Stack Using Singly Linked List](#)